

Promises og async/await

💡 Dagens fokus

- Forskellene på `.then()`/`.catch()` kæder og den nyere `async/await` syntaks
- Hvordan kan vi intercepte form submits i browseren og sende data til en server
- Hvordan kan det bruges i jeres chatbots?



Dagens program

1. Vis jeres Deck of Cards løsninger ved bordet
2. Præsenter eksempler og læring for klassen
3. Repetition: Fetch API & Promises
4. Async/Await Syntax
5. Hands-on: Konverter til Async/Await
6. Intercepting form submits
7. Hands-on: Opret form handler
8. Asynkron form data transmission
9. Hands-on: Send form data asynkront
10. Hands-on: Chatbot integration plan

👥 Grupper ⌚ 20 min

👥 Grupper ⌚ 20 min

🤝 Parvis ⌚ 20 min

👤 Solo ⌚ 20 min

👤 Solo ⌚ 25 min

🤝 Parvis ⌚ 10 min

1. Vis jeres Deck of Cards løsninger ved bordet

👥 Grupper ⌚ 20 min

💡 Find tilbage til jeres bord (I deles måske parvis)

Hver person (eller par, hvis I sidder ved samme bord) får **3 minutter** til at demonstrere deres løsning:

- **Vis** jeres kortspil i aktion
- **Forklar kort** jeres tilgang til at bruge Deck of Cards API'et
- **Fortæl** hvor I måtte slå op i dokumentation eller på anden måde søge hjælp
- **Highlight én udfordring** I stødte på og hvordan I løste den
- **Del én ting** I lærte om fetch/promises undervejs

Resten af bordet: Stil opklarende spørgsmål og notér interessante løsninger

2. Præsenter eksempler og læring for klassen

👥 Grupper ⌚ 20 min

💡 Hvert bord udvælger et eksempel til præsentation

Vælg det mest interessante eksempel ved bordet, som I vil dele med klassen. Måske den mest avancerede løsning, eller måske en interessant måde at strukturere koden, eller den største læring.

- Skaberne af den udvalgte løsningen **demonstrerer** den for klassen
- **Forklar kort** jeres tilgang til at bruge Deck of Cards API'et
- **Fortæl** hvor I måtte slå op i dokumentation eller på anden måde søge hjælp
- **Highlight én udfordring** I stødte på og hvordan I løste den
- Hvor savnede I mest **AI og autocomplete** hjælp undervejs?

Alle I andre: Stil opklarende spørgsmål og notér interessante løsninger

3. Repetition: Fetch API & Promises

💡 Nøglekoncepter:

Promises er JavaScript objekter der repræsenterer det eventuelle resultat af en asynkron operation:

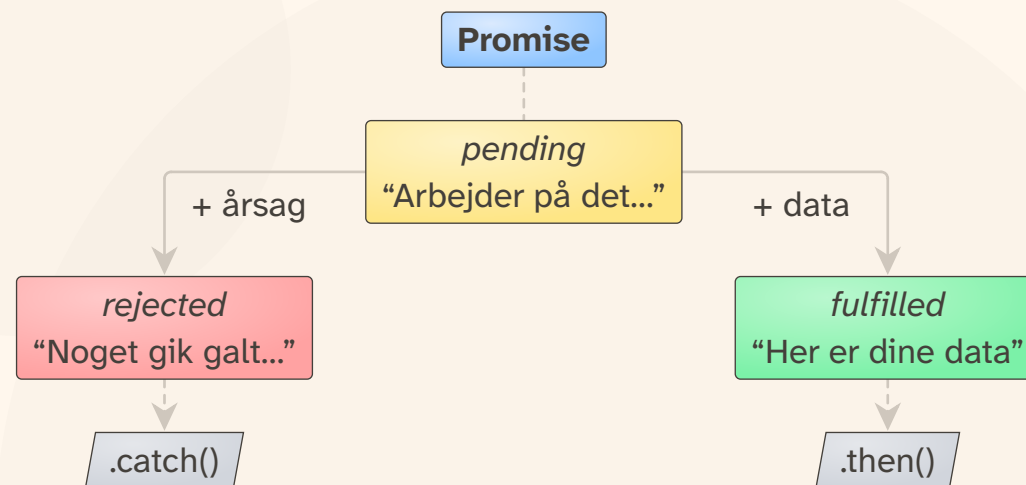
- Tre tilstande: **pending** (afventer), **fulfilled** (opfyldt), **rejected** (afvist)
- Gør asynkron kode mere læsbar og håndterbar

Fetch APIet bruges til at lave HTTP requests fra browseren:

```
fetch('https://api.example.com/data')  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error('Fejl:', error));
```

Asynkrone operationer er nødvendige for:

- HTTP requests til servere
- Database queries
- Filsystem operationer
- Operationer der tager uforudsigelig tid



4. Async/Await Syntax

Async/await er en nyere syntaks til at håndtere promises:

- **async** funktioner returnerer automatisk et Promise
- **await** pauser funktionen indtil et Promise er resolved
- Gør at asynkron kode læses som synkron kode
- Mere traditionel fejlhåndtering med **try/catch**

Grundlæggende syntaks:

```
async function fetchData() {  
  try {  
    const response = await fetch('https://api.example.com/data');  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error('Fejl:', error);  
  }  
}
```

4.1 Sammenligning: .then() vs async/await

🗨 Med din sidemakker: Forklar hinanden hver linje i disse eksempler:

Promise chain:

```
function fetchUserData() {  
  return fetch('/api/user')  
    .then(response => {  
      if (!response.ok) throw new Error('Failed');  
      return response.json();  
    })  
    .then(user => {  
      return fetch(`/api/posts/${user.id}`);  
    })  
    .then(posts => posts.json())  
    .catch(error => {  
      console.error('Error:', error);  
    });  
}
```

Med async/await:

```
async function fetchUserData() {  
  try {  
    const response = await fetch('/api/user');  
    if (!response.ok) throw new Error('Failed');  
    const user = await response.json();  
  
    const posts = await fetch(`/api/posts/${user.id}`);  
  
    return await posts.json();  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}
```

4.2 Async/Await i praksis

🔔 Vigtige regler og koncepter:

Async funktioner:

- Returnerer altid et Promise (også hvis du ikke eksplicit returnerer et)
- Man kan kun bruge **await** inde i **async** funktioner
- Arrow functions kan også være async: **const getData = async () => {}**

Error handling:

- Try/catch fanger både network fejl og HTTP fejl
- Uncaught errors i async funktioner bliver til rejected Promises
- Du kan kombinere **async/await** med **.catch()** hvis nødvendigt

Await keyword:

- Kan kun bruges med Promises eller “thenable” objekter
- Pauser funktionen indtil Promise resolves
- Returnerer resolved value eller kaster fejl hvis rejected

Almindelige fejl og løsninger:

- Glemmer **await**: Får Promise objekt i stedet for data
- Bruger **await** uden **async**: Syntax error
- Manglende error handling: Unhandled promise rejections

5. Hands-on: Konverter til Async/Await

🤝 Parvis ⌚ 20 min

Find sammen med din Deck of Cards makker og konverter jeres fetch kode fra `.then()/catch()` til `async/await` (eller omvendt, hvis I allerede har brugt `async/await`).

📄 Trin-for-trin konvertering:

- 01 Lav en kopi af filen for at bevare den originale version
- 02 Wrap din fetch kode i en `async function`
- 03 Erstat `.then()` med `await` keywords
- 04 Udskift `.catch()` med `try/catch` blokke
- 05 Test at begge versioner fungerer ens

💬 Diskussionspunkter mens I arbejder:

- Hvilken syntaks synes I er mest læsbar?
- Hvornår ville I vælge `.then()` frem for `async/await`?
- Hvordan håndteres fejl forskelligt i de to tilgange?
- Hvad synes I om forskellen på anonyme callback funktioner vs. midlertidige variabler?

```
fetch('https://api.example.com/data')  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error(error));
```



```
async function fetchData() {  
  try {  
    const response = await fetch('https://api.example.com/data');  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

6. Intercepting form submits

💡 Nøglekoncepter:

Form interception giver os kontrol over hvad der sker når brugere submitter formularer:

- Stopper browserens default navigation/reload-adfærd med `preventDefault()`
- Tillader asynkron data transmission samtidig med at UI state bevares (scroll position, input værdier, osv.)
- Giver mulighed for real-time validering og feedback op imod server-side kode
- Kernefunktionalitet i moderne single-page applications (SPA'er)

💬 Med din sidemakker – forklar på skift en linje ad gangen for hinanden:

```
const form = document.getElementById("chat-form");
form.addEventListener("submit", function(event) {
  console.log(event);
  event.preventDefault();
  const formData = new FormData(form);
  console.log(formData);
  const data = Object.fromEntries(formData);
  console.log(data);
});
```

Diskussionspunkter:

- 01 Forklar hvad hvert objekt og metode gør
- 02 Forklar hvornår events bliver triggeret
- 03 Forklar indholdet af dét, der logges i konsollen

6.1 Form event handling i detaljer

Submit vs Click events:

- **click** event affyres kun når der klikkes på en specifik knap
- **submit** event affyres når formen submittes (Enter key, submit knap, etc.)
- **submit** er mere robust og fanger alle submission metoder
- Brug **submit** for form handling, **click** for specifikke knap-handlinger

💡 Adgang til form data - forskellige metoder:

Metode 1: FormData API (anbefalet)

```
const formData = new FormData(form);  
const username = formData.get('username');  
const email = formData.get('email');
```

Metode 3: querySelector

```
const username = form.querySelector('input[name="username"]').value;
```

preventDefault() timing:

- Skal kaldes tidligt i event handleren
- Stopper browserens default behavior permanent
- Kan ikke fortrydes efter den er kaldt
- Påvirker ikke andre event listeners på samme element

Metode 2: Direkte element adgang

```
const username = form.elements.username.value;  
const email = form.elements.email.value;
```

6.2 Client-side form validering

💡 Et simpelt eksempel:

```
form.addEventListener('submit', function(event) {
  event.preventDefault();
  const formData = new FormData(form);
  const data = Object.fromEntries(formData);
  // Validering
  if (!data.email || !data.email.includes('@')) {
    showError('Ugyldig email adresse');
    return;
  }
  if (data.message.length < 10) {
    showError('Besked skal være mindst 10 tegn');
    return;
  }
  // Send data hvis validering passerer
  sendFormData(data);
});
```

Best practices for form validering:

- Giv brugeren øjeblikkelig feedback
- Vis specifikke fejlmeddelelser
- Bevar brugerens input ved fejl
- Validér på både client og server side

7. Hands-on: Opret form handler

🚩 Solo ⌚ 20 min

💬 Byg en form submit handler i din chatbot

Trin-for-trin instruktioner:

- 01 Skriv JavaScript der kan intercepte formens **submit** event
- 02 Log form data til konsollen for at teste funktionaliteten

Eksempel form struktur:

```
<form id="chat-form">
  <textarea name="message" placeholder="Besked"></
textarea>
  <button type="submit">Send</button>
  <div id="error-message" style="color: red;"></div>
</form>
```

- 01 Implementér simpel client-side validering
- 02 Vis fejlmeddelelser til brugeren ved valideringsfejl

Validerings-eksempel:

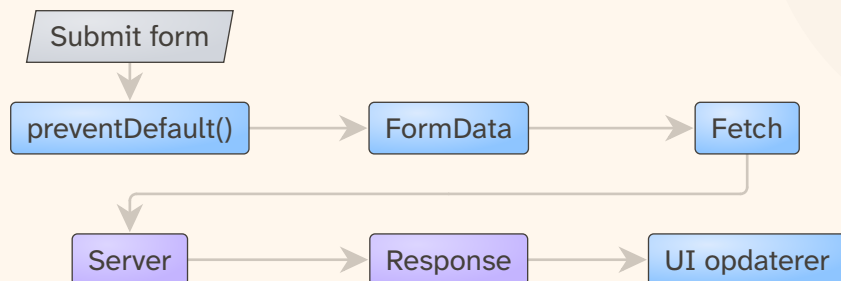
```
if (formData.get("message").length < 10) {
  showError('Besked skal være mindst 10 tegn');
  return;
}
```

8. Asynkron form data transmission

💡 Nøglekoncepter:

Asynkron form transmission kombinerer form interception med Fetch API:

- Forhindrer standard page reload og bevarer UI state
- Sender data til serveren i baggrunden mens brugeren venter
- Giver mulighed for dynamisk UI opdatering og feedback
- Grundlæggende byggesten i moderne web apps



📦 Eksempel:

```
form.addEventListener('submit',  
  async function(event) {  
    event.preventDefault();  
    const formData = new FormData(form);  
  
    try {  
      const response = await fetch('/api/contact', {  
        method: 'POST',  
        body: formData  
      });  
      const result = await response.json();  
      handleSuccess(result);  
    } catch (error) {  
      handleError(error);  
    }  
  });
```

8.1 Data serialization tilgange

🔗 Forskellige måder at sende form data:

1. FormData (multipart/form-data)

```
const formData = new FormData(form);  
const response = await fetch('/api/endpoint', {  
  method: 'POST',  
  body: formData  
});
```

- Understøtter fil uploads
- Direkte kompatibel med HTML forms
- Browseren sætter automatisk Content-Type header

💡 Valg af tilgang:

- Brug FormData til simple forms og file uploads
- Brug JSON til komplekse datastrukturer og REST API'er
- Server-side kode skal understøtte det format, du sender

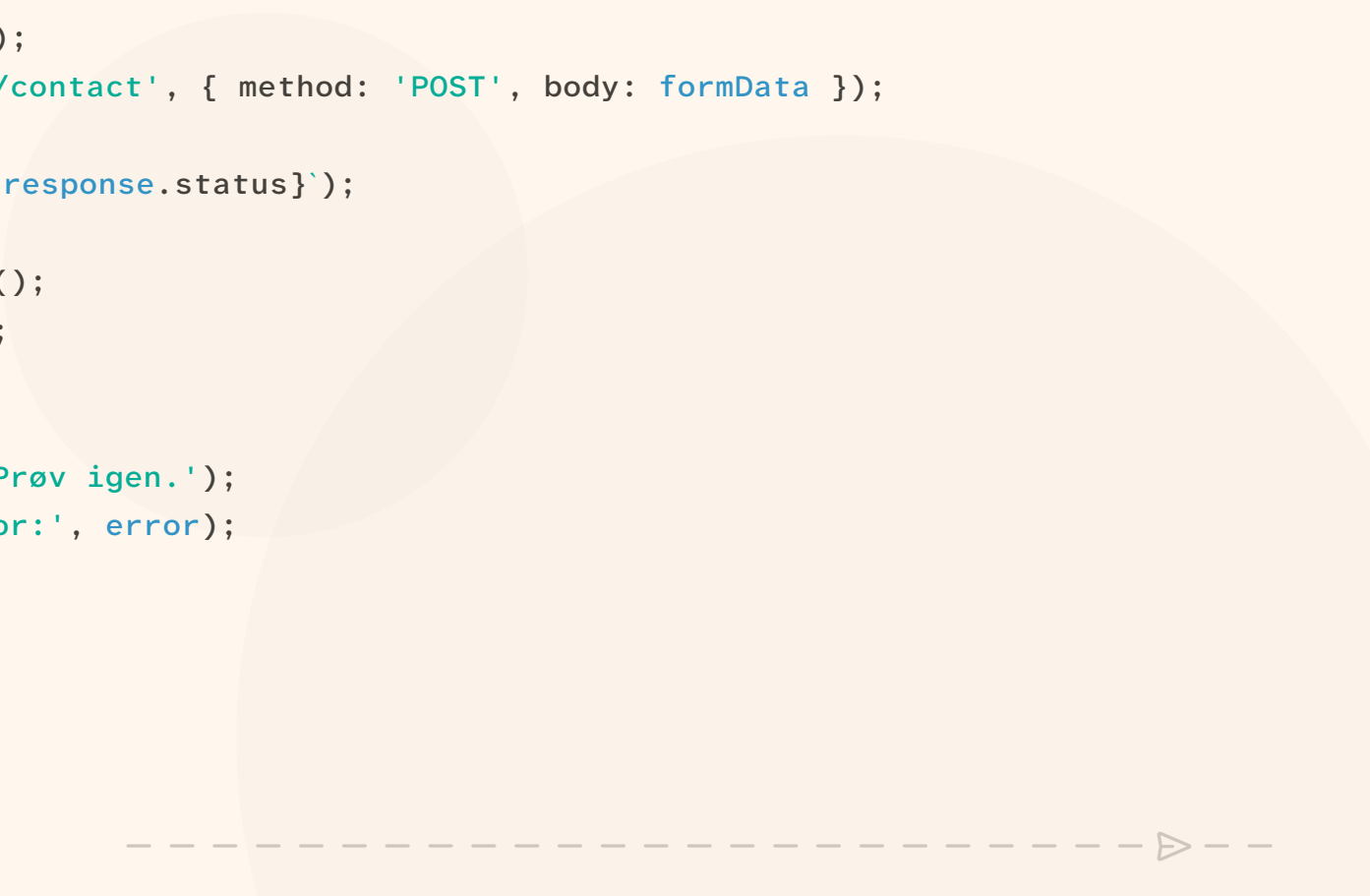
2. JSON payload

```
const formData = new FormData(form);  
const data = Object.fromEntries(formData);  
const response = await fetch('/api/endpoint', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify(data)  
});
```

- Kræver manuel serialization
- Forventes/foretrækkes af mange REST API'er

8.2 Server response handling

```
async function submitForm(form) {  
  try {  
    showLoadingState();  
    const formData = new FormData(form);  
    const response = await fetch('/api/contact', { method: 'POST', body: formData });  
    if (!response.ok) {  
      throw new Error(`Server error: ${response.status}`);  
    }  
    const result = await response.json();  
    showSuccessMessage(result.message);  
    form.reset();  
  } catch (error) {  
    showErrorMessage('Noget gik galt. Prøv igen.');
```



```
    console.error('Form submission error:', error);  
  } finally {  
    hideLoadingState();  
  }  
}
```

9. Hands-on: Send form data asynkront

🕒 Solo ⌚ 25 min

Udvid din form handler fra forrige øvelse til at sende data asynkront til serveren.

Trin-for-trin instruktioner:

- 01 Modificér din form handler til at bruge Fetch API i stedet for bare at logge data
- 02 Sæt et mock API endpoint op i din chatbot Express server (f.eks. **POST /api/test**)
- 03 Send form data asynkront til dit endpoint
- 04 Vis server response på siden for brugeren
- 05 Implementer både success og error states med brugervenlig feedback

Server-side setup (Express):

```
app.post('/api/test', (req, res) => {  
  console.log('Modtaget data:', req.body);  
  res.json({  
    success: true,  
    message: 'Data modtaget succesfuldt',  
    receivedData: req.body  
  });  
});
```

Tips til implementering:

- Test først at serveren kører og kan modtage requests
- Brug DevTools Network tab til at se dine HTTP requests
- Start simpelt og byg kompleksiteten gradvist op
- Prøv at lave server-side validering (så serveren svarer med en status **400** ved ugyldig data)

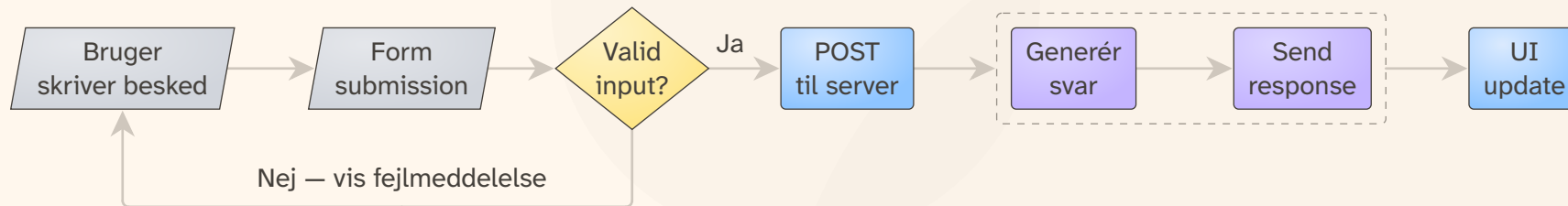
10. Hands-on: Chatbot integration plan

Parvis ⌚ 10 min

💡 Overvej hvordan dagens emner kan integreres in din chatbot

Frontend implementering:

- Hvilken feedback vil du give brugeren mens serveren tænker?
- Hvordan vil du vise chat historik?
- Hvilke validerings-regler har du brug for?



Backend integration:

- Hvilke API endpoints skal du oprette?
- Hvordan vil du strukturere dine request/response data?
- Hvilken error handling er vigtig for din use case?

💬 Diskutér med din sidemakker:

- Vil du bruge FormData eller JSON til server kommunikation?
- Hvordan balancerer du mellem client-side og server-side validering?
- Hvor hurtigt forventer brugerne svar fra din chatbot?
- Hvordan kommunikerer du systemets begrænsninger til brugeren?

